

02 — System Architecture

02 — System Architecture

Revision: 1 **Last modified:** 2026-07-05T14:20:00Z **Status:** Draft
— consolidated from SPECIFICATION.md §5–§6, 01-data-plane.md §0,
02-control-plane.md §0, and 05-repo-layout-tooling-and-helix-ecosystem.md.

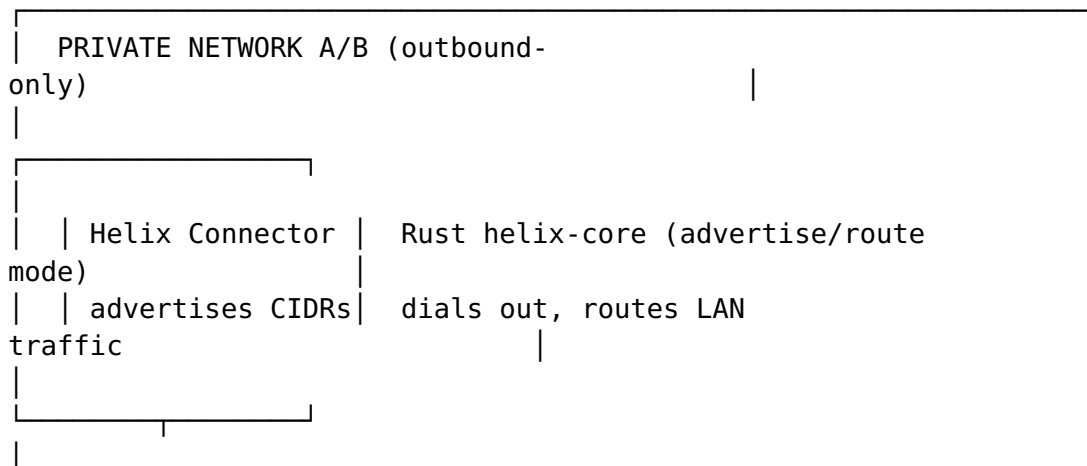
1. One-screen architecture overview

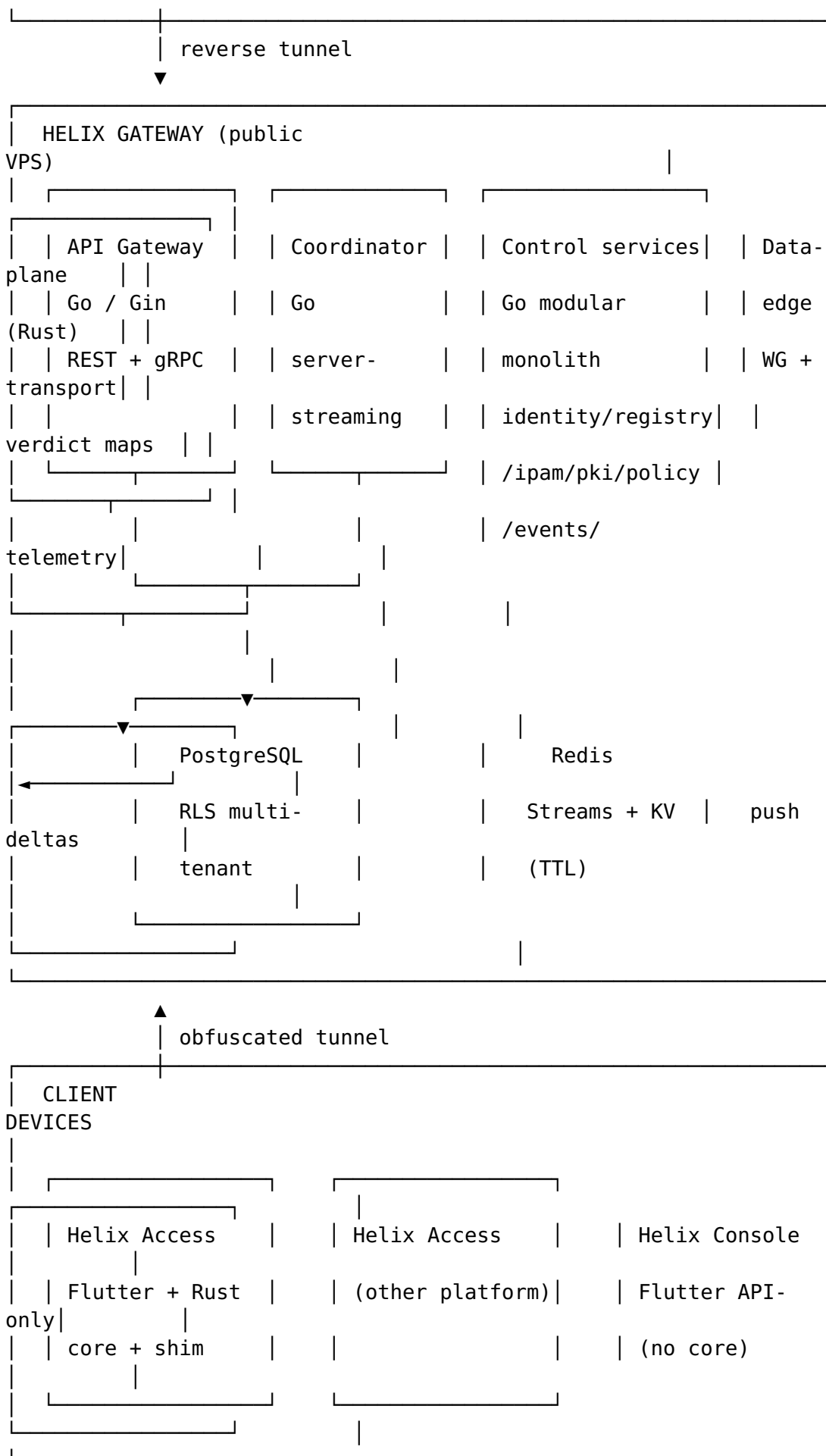
HelixVPN is a **three-role, hub-and-spoke overlay**:

- **Connector** dials outbound from a private network and advertises CIDRs.
- **Gateway** (public VPS) hosts the control plane (Go) and data-plane edge (Rust).
- **Client** dials outbound from an end-user device to reach authorized networks or a privacy exit.

The control plane **never sits in the packet path**. If it fails, existing tunnels keep forwarding (**fail-static**).

2. C4-style component view





3. Data-plane layering

L4 Application traffic (user packets / advertised LAN hosts)
L3 WireGuard (Noise IK, Curve25519, ChaCha20-Poly1305) – CRYPTO CORE
L2.5 (optional) DAITA shaping: pad + cover traffic (maybe not) – ABOVE WG
L2 Pluggable transport (exactly one active):

- plain UDP
- Shadowsocks-wrap
- LW0
- MASQUE CONNECT-UDP / H3
- UDP-over-TCP
- port-hop / 443 / 53 disguise

L1 IP to gateway public endpoint (:443/udp, :443/tcp, :51820/udp, ...)

The transport layer **only changes how encrypted WG datagrams look on the wire.**

4. Reuse map

Three reuse pillars:

Pillar	Shared by	Repo (future)
Rust core (helix-core)	Client, Connector, Gateway edge	vasic-digital/helix_core
Flutter UI (helix-ui)	Access, Connector, Console	vasic-digital/helix_ui
Schema-generated clients (helix-proto)	Dart, Go, Rust	vasic-digital/helix_proto

helix-core (Rust)

```
├─ helix-transport ← obfuscation carriers (shared client↔edge)
├─ helix-wg        ← boringtun wrapper
├─ helix-ffi       ← flutter_rust_bridge + UniFFI
├─ helix-core      ← orchestrator + reconciler
└─ helix-route     ← overlay addressing + verdict maps
```

helix-ui (Flutter/Dart)

```
├─ helix_design ← OpenDesign system (decoupled submodule)
├─ app_access   ← consumer VPN flavor
├─ app_connector ← network-operator flavor
└─ app_console  ← admin flavor (no core_ffi)
```

helix-proto

```
└─ .proto / OpenAPI → generated Dart/Go/Rust clients
```

5. Repository & component layout

```
helixvpn/                                # umbrella
├─ helix_core/                           # Rust workspace
│   ├── crates/helix_transport/
│   ├── crates/helix_wg/
│   ├── crates/helix_ffi/
│   ├── crates/helix_core/
│   ├── crates/helix_tun/
│   ├── crates/helix_route/
│   └── crates/helix_daita/
├─ helix_edge/                           # Gateway data-plane edge
├─ helix_go/                             # Go control plane (modular
monolith)
├─ helix_proto/                          # Protobuf + OpenAPI
├─ helix_ui/                             # Flutter monorepo
│   ├── app_access/
│   ├── app_connector/
│   └── app_console/
├─ shims/                                # apple / android / windows /
linux / harmonyos / aurora
├─ deploy/                               # Podman quadlets, Terraform,
Grafana-as-code, helixvpctl
├─ submodules/                           # containers, helix_qa,
challenges, docs_chain, security, vision_engine, ...
```

6. Cross-cutting contracts

Three contracts bind every component:

1. **Transport trait** (Rust) — owning doc v02-data-plane/transport-trait.md. One implementation, three consumers.
2. **WatchNetworkMap stream** (protobuf) — owning doc v03-control-plane/svc-coordinator.md. Snapshot + delta stream of desired state.
3. **FFI surface** (helix-ffi) — owning doc v04-client/ffi-surface.md. Dart drives the core and consumes a status stream.

7. Cross-references

- Data-plane details → [03 — Data Plane](#)
 - Control-plane details → [04 — Control Plane](#)
 - Client core & UI → [05 — Client Core & UI](#)
 - API contracts → [08 — API Contracts](#)
 - Infrastructure & deploy → [07 — Infrastructure & DevOps](#)
-

Sources: docs/research/mvp/final/SPECIFICATION.md §5-§6, 01-data-plane.md §0, 02-control-plane.md §0, 05-repo-layout-tooling-and-helix-ecosystem.md §0-§2.